

# Logistics

# About this course

- This course about
  - Many languages (the nerdy kind)
  - Compilers
  - Interpreters
  - Many related concepts and techniques

# About this course

- This course about
  - Many languages (the monkey kind)
  - Compilers
  - Interpreters
  - Many related topics



## Learning goals and competences

- Compilers are a melting-put of various fields of computer science
  - Formal languages and complexity
  - Data structures
  - Graph algorithms
  - Computer architecture
  - Engineering

## Learning goals and competences

- Compilers are a melting-put of various fields of computer science
  - Formal languages and complexity
  - Data structures
  - Graph algorithms
  - Computer architecture
  - Engineering
- Compilers are HUGE projects (LLVM has  $\approx 13.5$  MLoc +  $\approx 9$ M lines comments)
  - Software engineering
  - Traditions and best practices

## Learning goals and competences

- Compilers are a melting-put of various fields of computer science
  - Formal languages and complexity
  - Data structures
  - Graph algorithms
  - Computer architecture
  - Engineering
- Compilers are HUGE projects (LLVM has  $\approx 13.5$  MLoc +  $\approx 9$ M lines comments)
  - Software engineering
  - Traditions and best practices
- Hands-on! **We** will implement multiple
  - Compilers
  - Interpreters
  - Language tools

# Logistics

- You can reach me by
  - by email [penschuck@imada.sdu.dk](mailto:penschuck@imada.sdu.dk)
  - in my office Ø16-601b-2 (just come by ... I might even be there ;))

# Logistics

- You can reach me by
  - by email [penschuck@imada.sdu.dk](mailto:penschuck@imada.sdu.dk)
  - in my office Ø16-601b-2 (just come by ... I might even be there ;))
  - do **not** write messages on itslearning to me (not monitored)
  - do **not** call me on the phone (not monitored)
- 5 ECTS course. Expect  $\approx 150$ h of work.
  - $\approx 40$ h for lectures and tutorials
  - $\approx 10$ h for pre/post processing of classes

# Logistics

- You can reach me by
  - by email [penschuck@imada.sdu.dk](mailto:penschuck@imada.sdu.dk)
  - in my office Ø16-601b-2 (just come by ... I might even be there ;))
  - do **not** write messages on itslearning to me (not monitored)
  - do **not** call me on the phone (not monitored)
- 5 ECTS course. Expect  $\approx 150$ h of work.
  - $\approx 40$ h for lectures and tutorials
  - $\approx 10$ h for pre/post processing of classes
  - $\approx 100$ h of working on problems/projects

## Logistics

- You can reach me by
  - by email [penschuck@imada.sdu.dk](mailto:penschuck@imada.sdu.dk)
  - in my office Ø16-601b-2 (just come by ... I might even be there ;))
  - do **not** write messages on itslearning to me (not monitored)
  - do **not** call me on the phone (not monitored)
- 5 ECTS course. Expect  $\approx 150$ h of work.
  - $\approx 40$ h for lectures and tutorials
  - $\approx 10$ h for pre/post processing of classes
  - $\approx 100$ h of working on problems/projects
- This is new course: things will go wrong. **Let me know when they do.**

## Logistics

- You can reach me by
  - by email [penschuck@imada.sdu.dk](mailto:penschuck@imada.sdu.dk)
  - in my office Ø16-601b-2 (just come by ... I might even be there ;))
  - do **not** write messages on itslearning to me (not monitored)
  - do **not** call me on the phone (not monitored)
- 5 ECTS course. Expect  $\approx 150$ h of work.
  - $\approx 40$ h for lectures and tutorials
  - $\approx 10$ h for pre/post processing of classes
  - $\approx 100$ h of working on problems/projects
- This is new course: things will go wrong. **Let me know when they do.**
- **PLEASE, let me know if I can make your live easier ;)**

## Problems classes & Projects

- Teaching Assistant: **Carl-Gustav Øboe Rasmussen**
- There are (initially) weekly problem sheets
  - Problems and solutions are discussed during *tutorials*

## Problems classes & Projects

- Teaching Assistant: **Carl-Gustav Øboe Rasmussen**
- There are (initially) weekly problem sheets
  - Problems and solutions are discussed during *tutorials*
  - Work on these problems in groups (preferred!) or alone

## Problems classes & Projects

- Teaching Assistant: **Carl-Gustav Øboe Rasmussen**
- There are (initially) weekly problem sheets
  - Problems and solutions are discussed during *tutorials*
  - Work on these problems in groups (preferred!) or alone
  - Come prepared: Problems are designed such that you learn while **actively** working on them. Passively absorbing them won't have the same effect.

## Problems classes & Projects

- Teaching Assistant: **Carl-Gustav Øboe Rasmussen**
- There are (initially) weekly problem sheets
  - Problems and solutions are discussed during *tutorials*
  - Work on these problems in groups (preferred!) or alone
  - Come prepared: Problems are designed such that you learn while **actively** working on them. Passively absorbing them won't have the same effect.
- There are **two mandatory** projects.
  - You can only pass this course, if you passed both of them.
  - Projects have to be submitted in groups.

## Problems classes & Projects

- Teaching Assistant: **Carl-Gustav Øboe Rasmussen**
- There are (initially) weekly problem sheets
  - Problems and solutions are discussed during *tutorials*
  - Work on these problems in groups (preferred!) or alone
  - Come prepared: Problems are designed such that you learn while **actively** working on them. Passively absorbing them won't have the same effect.
- There are **two mandatory** projects.
  - You can only pass this course, if you passed both of them.
  - Projects have to be submitted in groups.
- Some of the questions on the problems sheets will be help you with the projects.

The code you write in this course is not a product!

## The code you write in this course is not a product!

So why should you measure your efficiency as the *time to produce* this code?

- Instead the **goal** for **you**
  - to recap and better understand the concepts from lectures
  - to apply these concepts (*writing code* is just a very good tool to do that!)
  - experience pitfalls and limitations first hand
  - learn and practice how to manage a large project yourself
- First learn how to do foundational things yourself.
- Only then use tools to accelerate things.

## GenAI: The rules until further notice

- I **strongly** recommend not using GenAI to produce text or code for this course.
- I will give written feedback (comments how to improve), only on human written submissions.
- Asking questions if you get stuck, may be a good use.
- If you use **any** GenAI for submissions, you have to attach
  - SDU's AI declaration form
  - the **complete** dialog
- Failing to do so constitutes an exam irregularity.

most of the theory we do is covered in:



## **Engineering a Compiler**

by Keith D. Cooper and Linda Torczon

*be careful:* many search engines directly point you to unofficial copies on github (etc.); these are likely copyright infringements.

many of the implementation details are inspired by:



## **Essentials of Compilation – An Incremental Approach in Python**

by J.G. Siek

**free PDF available**

# Material

- Everything will be published (or linked to) in itslearning.
- I'm experimenting with recording this course, but **no promises**.
- I will share my notes with you.

## Prerequisites

- Assembly (mostly DM548/DM588):
  - basics of the x86 machine (basic instructions, addressing modes etc.)
  - usage of GNU `as` (assembler) and GNU `ld` (linker)

## Prerequisites

- Assembly (mostly DM548/DM588):
  - basics of the x86 machine (basic instructions, addressing modes etc.)
  - usage of GNU as (assembler) and GNU ld (linker)
- Python 3.10 (or higher)
  - We'll be implementing most code in Python!
  - Some understanding of OOP (mostly inheritance and Python's super keyword)

## Prerequisites

- Assembly (mostly DM548/DM588):
  - basics of the x86 machine (basic instructions, addressing modes etc.)
  - usage of GNU `as` (assembler) and GNU `ld` (linker)
- Python 3.10 (or higher)
  - We'll be implementing most code in Python!
  - Some understanding of OOP (mostly inheritance and Python's `super` keyword)
- Git
  - We'll use <https://gitlab.sdu.dk> for tutorials and projects

## Prerequisites

- Assembly (mostly DM548/DM588):
  - basics of the x86 machine (basic instructions, addressing modes etc.)
  - usage of GNU `as` (assembler) and GNU `ld` (linker)
- Python 3.10 (or higher)
  - We'll be implementing most code in Python!
  - Some understanding of OOP (mostly inheritance and Python's `super` keyword)
- Git
  - We'll use <https://gitlab.sdu.dk> for tutorials and projects

We'll recap advanced concepts, but I expect you to lookup things you learned in previous courses.